

After three attempts, I've finally found a suitable dataset!

This first dataset that I chose turned out to be bogus - it was synthetically created and didn't give any information about where it came from.

The second dataset came from an identified source; however, it was too simple. Incredibly easy to train models with almost perfect accuracy.

This third dataset comes from a recognised source [1]. This dataset was more challenging and allowed me to use some of the methods learned on the Professional Certificate course.

[1] Davide Anguita, Alessandro Ghio, Luca Oneto, Xavier Parra and Jorge L. Reyes-Ortiz. A Public Domain Dataset for Human Activity Recognition Using Smartphones. 21st European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning, ESANN 2013. Bruges, Belgium 24-26 April 2013.

<https://www.kaggle.com/code/levkharlashkin/human-activity-recognition?select=test.csv>

```
In [961... # The first step is to import some libraries

import numpy as np # Linear algebra
import pandas as pd # data processing

from sklearn import metrics #Import scikit-learn metrics module for accuracy cal

from sklearn.metrics import classification_report
```

```
In [963... # I now read the csv file and make a copy
X_train = pd.read_csv("human_activity/train.csv")
X_test = pd.read_csv("human_activity/test.csv")
```

```
In [964... # Examining the dataframe...
X_train
```

Out[964...

	tBodyAcc-mean()-X	tBodyAcc-mean()-Y	tBodyAcc-mean()-Z	tBodyAcc-std()-X	tBodyAcc-std()-Y	tBodyAcc-std()-Z	tBodyAcc-mad()-X
0	0.288585	-0.020294	-0.132905	-0.995279	-0.983111	-0.913526	-0.995112
1	0.278419	-0.016411	-0.123520	-0.998245	-0.975300	-0.960322	-0.998807
2	0.279653	-0.019467	-0.113462	-0.995380	-0.967187	-0.978944	-0.996520
3	0.279174	-0.026201	-0.123283	-0.996091	-0.983403	-0.990675	-0.997099
4	0.276629	-0.016570	-0.115362	-0.998139	-0.980817	-0.990482	-0.998321
...	...	...	...	...	...	...	...
7347	0.299665	-0.057193	-0.181233	-0.195387	0.039905	0.077078	-0.282301
7348	0.273853	-0.007749	-0.147468	-0.235309	0.004816	0.059280	-0.322552
7349	0.273387	-0.017011	-0.045022	-0.218218	-0.103822	0.274533	-0.304515
7350	0.289654	-0.018843	-0.158281	-0.219139	-0.111412	0.268893	-0.310487
7351	0.351503	-0.012423	-0.203867	-0.269270	-0.087212	0.177404	-0.377404

7352 rows × 563 columns



In [965...

X_test

Out[965...

	tBodyAcc-mean()-X	tBodyAcc-mean()-Y	tBodyAcc-mean()-Z	tBodyAcc-std()-X	tBodyAcc-std()-Y	tBodyAcc-std()-Z	tBodyAcc-mad()-X
0	0.257178	-0.023285	-0.014654	-0.938404	-0.920091	-0.667683	-0.952501
1	0.286027	-0.013163	-0.119083	-0.975415	-0.967458	-0.944958	-0.986799
2	0.275485	-0.026050	-0.118152	-0.993819	-0.969926	-0.962748	-0.994403
3	0.270298	-0.032614	-0.117520	-0.994743	-0.973268	-0.967091	-0.995274
4	0.274833	-0.027848	-0.129527	-0.993852	-0.967445	-0.978295	-0.994111
...	...	...	...	...	...	...	...
2942	0.310155	-0.053391	-0.099109	-0.287866	-0.140589	-0.215088	-0.356083
2943	0.363385	-0.039214	-0.105915	-0.305388	0.028148	-0.196373	-0.373540
2944	0.349966	0.030077	-0.115788	-0.329638	-0.042143	-0.250181	-0.388017
2945	0.237594	0.018467	-0.096499	-0.323114	-0.229775	-0.207574	-0.392380
2946	0.153627	-0.018437	-0.137018	-0.330046	-0.195253	-0.164339	-0.430974

2947 rows × 563 columns



In [966...

y_train = X_train.Activity

In [971...

X_train = X_train.drop("Activity", axis='columns')


```
In [979... y_test = X_test.Activity
```

```
In [981... X_test = X_test.drop("Activity", axis='columns')
```

```
In [983... X_test
```

```
Out[983...      tBodyAcc-  tBodyAcc-  tBodyAcc-  tBodyAcc-  tBodyAcc-  tBodyAcc-  tBodyAcc-
      mean()-X   mean()-Y   mean()-Z   std()-X     std()-Y     std()-Z     mad()-X
0      0.257178 -0.023285 -0.014654 -0.938404 -0.920091 -0.667683 -0.952501
1      0.286027 -0.013163 -0.119083 -0.975415 -0.967458 -0.944958 -0.986799
2      0.275485 -0.026050 -0.118152 -0.993819 -0.969926 -0.962748 -0.994403
3      0.270298 -0.032614 -0.117520 -0.994743 -0.973268 -0.967091 -0.995274
4      0.274833 -0.027848 -0.129527 -0.993852 -0.967445 -0.978295 -0.994111
...      ...      ...      ...      ...      ...      ...      ...
2942    0.310155 -0.053391 -0.099109 -0.287866 -0.140589 -0.215088 -0.356083
2943    0.363385 -0.039214 -0.105915 -0.305388  0.028148 -0.196373 -0.373540
2944    0.349966  0.030077 -0.115788 -0.329638 -0.042143 -0.250181 -0.388017
2945    0.237594  0.018467 -0.096499 -0.323114 -0.229775 -0.207574 -0.392380
2946    0.153627 -0.018437 -0.137018 -0.330046 -0.195253 -0.164339 -0.430974
```

2947 rows × 562 columns



```
In [985... y_test
```

```
Out[985... 0      STANDING
1      STANDING
2      STANDING
3      STANDING
4      STANDING
...
2942    WALKING_UPSTAIRS
2943    WALKING_UPSTAIRS
2944    WALKING_UPSTAIRS
2945    WALKING_UPSTAIRS
2946    WALKING_UPSTAIRS
Name: Activity, Length: 2947, dtype: object
```

```
In [987... # Any missing data?
X_test.isnull().sum()
```

```
Out[987... tBodyAcc-mean()-X      0
tBodyAcc-mean()-Y      0
tBodyAcc-mean()-Z      0
tBodyAcc-std()-X       0
tBodyAcc-std()-Y       0
..
angle(tBodyGyroJerkMean,gravityMean) 0
angle(X,gravityMean)      0
angle(Y,gravityMean)      0
angle(Z,gravityMean)      0
subject                   0
Length: 562, dtype: int64
```

In []:

```
In [990... from sklearn import preprocessing

min_max_scaler = preprocessing.MinMaxScaler()
X_train = min_max_scaler.fit_transform(X_train)
X_train = pd.DataFrame(X_train)

X_test = min_max_scaler.fit_transform(X_test)
X_test = pd.DataFrame(X_test)
```

In [992... X_train

```
Out[992...      0      1      2      3      4      5      6      7
0  0.644292  0.489853  0.433547  0.002361  0.008748  0.043237  0.002444  0.008546  0
1  0.639209  0.491795  0.438240  0.000877  0.012824  0.019839  0.000596  0.012749  0
2  0.639827  0.490266  0.443269  0.002310  0.017058  0.010528  0.001740  0.018464  0
3  0.639587  0.486900  0.438359  0.001954  0.008596  0.004662  0.001450  0.008767  0
4  0.638314  0.491715  0.442319  0.000931  0.009945  0.004759  0.000839  0.010331  0
...      ...      ...      ...      ...      ...      ...      ...      ...
7347  0.649833  0.471403  0.409383  0.402307  0.542650  0.538539  0.358850  0.530383  0
7348  0.636926  0.496125  0.426266  0.382346  0.524338  0.529640  0.338724  0.493247  0
7349  0.636694  0.491495  0.477489  0.390891  0.467640  0.637266  0.347742  0.457948  0
7350  0.644827  0.490578  0.420860  0.390430  0.463679  0.634447  0.344756  0.473556  0
7351  0.675752  0.493788  0.398066  0.365365  0.476309  0.588702  0.311298  0.488560  0
```

7352 rows × 562 columns



In [994... X_test

Out[994...

	0	1	2	3	4	5	6	7	
0	0.671879	0.557643	0.524648	0.041779	0.039955	0.222530	0.032601	0.037334	0
1	0.694704	0.574264	0.427079	0.016514	0.016271	0.036272	0.008768	0.015757	0
2	0.686363	0.553102	0.427948	0.003951	0.015037	0.024322	0.003484	0.014590	0
3	0.682260	0.542325	0.428538	0.003320	0.013366	0.021404	0.002879	0.012722	0
4	0.685848	0.550151	0.417320	0.003928	0.016277	0.013878	0.003687	0.016981	0
...	...	...	...	...	...	...	...	...	...
2942	0.713794	0.508207	0.445740	0.485861	0.429705	0.526560	0.447047	0.425588	0
2943	0.755910	0.531487	0.439381	0.473900	0.514074	0.539131	0.434917	0.484960	0
2944	0.745294	0.645268	0.430157	0.457346	0.478929	0.502986	0.424857	0.433347	0
2945	0.656384	0.626202	0.448179	0.461799	0.385112	0.531607	0.421825	0.360167	0
2946	0.589949	0.565605	0.410321	0.457067	0.402373	0.560650	0.395006	0.390826	0

2947 rows × 562 columns



In [996...

```
y_train.to_frame()
y_test.to_frame()
```

Out[996...

	Activity
0	STANDING
1	STANDING
2	STANDING
3	STANDING
4	STANDING
...	...
2942	WALKING_UPSTAIRS
2943	WALKING_UPSTAIRS
2944	WALKING_UPSTAIRS
2945	WALKING_UPSTAIRS
2946	WALKING_UPSTAIRS

2947 rows × 1 columns

In [998...

```
y_train = y_train.reset_index()
y_test = y_test.reset_index()
y_train = y_train.drop("index", axis='columns')
y_test = y_test.drop("index", axis='columns')
```

In [100... y_test

Out[100...

Activity	
0	STANDING
1	STANDING
2	STANDING
3	STANDING
4	STANDING
...	...
2942	WALKING_UPSTAIRS
2943	WALKING_UPSTAIRS
2944	WALKING_UPSTAIRS
2945	WALKING_UPSTAIRS
2946	WALKING_UPSTAIRS

2947 rows × 1 columns

```
In [100... # Find the unique numbers from the train labels
classes = np.unique(y_train)
nClasses = len(classes)
print('Total number of outputs : ', nClasses)
print('Output classes : ', classes)
```

Total number of outputs : 6

Output classes : ['LAYING' 'SITTING' 'STANDING' 'WALKING' 'WALKING_DOWNSTAIRS' 'WALKING_UPSTAIRS']

```
In [100... target_names = ['Walking', 'Walking Upstairs', 'Walking Downstairs', 'Sitting',
```

```
In [ ]: print('Training data shape:', X_train.shape, y_train.shape)
print('Testing data shape:', X_test.shape, y_test.shape)
```

PCA

```
In [112... #Dimensionality Reduction
from sklearn.decomposition import PCA

pca = PCA(n_components=40)
X_train_pc = pca.fit_transform(X_train)
X_test_pc = pca.transform(X_test)
explained_variance = pca.explained_variance_ratio_
print(pd.DataFrame(explained_variance))

print(sum(pca.explained_variance_ratio_))
```

```
0
0 0.615828
1 0.050640
2 0.042643
3 0.018626
4 0.016604
5 0.012495
6 0.011697
7 0.010868
8 0.009834
9 0.008507
10 0.008370
11 0.007051
12 0.006703
13 0.005993
14 0.005412
15 0.004889
16 0.004738
17 0.004560
18 0.004240
19 0.004112
20 0.004092
21 0.003869
22 0.003706
23 0.003532
24 0.003417
25 0.003249
26 0.003109
27 0.003044
28 0.002983
29 0.002885
30 0.002804
31 0.002705
32 0.002627
33 0.002561
34 0.002387
35 0.002372
36 0.002251
37 0.002177
38 0.002078
39 0.002057
0.9117156861660729
```

MODEL TRAINING

Decision Tree

```
In [112... # Load Libraries
from sklearn.tree import DecisionTreeClassifier # Import Decision Tree Classifier

# Create Decision Tree classifier object
clf = DecisionTreeClassifier(random_state=16)

# Train Decision Tree Classifier
clf = clf.fit(X_train_pc, y_train)
```

```
#Predict the response for test dataset
y_pred = clf.predict(X_test_pc)

# Model Accuracy, how often is the classifier correct?
print("Accuracy:", metrics.accuracy_score(y_test, y_pred))

print(classification_report(y_test, y_pred, target_names=target_names))
```

Accuracy: 0.7343060739735324

	precision	recall	f1-score	support
Walking	0.98	0.98	0.98	537
Walking Upstairs	0.64	0.79	0.71	491
Walking Downstairs	0.77	0.61	0.68	532
Sitting	0.80	0.58	0.67	496
Standing	0.53	0.84	0.65	420
Laying	0.80	0.60	0.68	471
accuracy			0.73	2947
macro avg	0.75	0.73	0.73	2947
weighted avg	0.76	0.73	0.74	2947

Decision Tree hyperparameter tuning: Grid Search

```
In [112... # Create the parameter grid based on the results of random search
params = {
    'max_depth': [2, 3, 4, 5, 6, 7, 8, 9, 10, 15, 20, 25],
    'min_samples_leaf': [5, 6, 7, 8, 9, 10, 15, 20, 25],
    'criterion': ["gini", "entropy"]
}

from sklearn.model_selection import GridSearchCV

# Instantiate the grid search model
grid_search = GridSearchCV(estimator=clf,
                           param_grid=params,
                           cv=4, n_jobs=-1, verbose=1, scoring = "accuracy")

grid_search.fit(X_train_pc, y_train)
```

Fitting 4 folds for each of 216 candidates, totalling 864 fits

```
Out[112... GridSearchCV ⓘ ?
  ▸ estimator: DecisionTreeClassifier
    ▸ DecisionTreeClassifier ⓘ
```

```
In [113... score_df = pd.DataFrame(grid_search.cv_results_)
```

```
In [113... grid_search.best_estimator_
```

Out[113...

```
DecisionTreeClassifier
DecisionTreeClassifier(criterion='entropy', max_depth=7, min_samples_leaf=20,
                      random_state=16)
```

In [113...

```
grid_search.best_score_
```

Out[113...

```
0.7861806311207835
```

In [119...

```
# Create Decision Tree classifier object
clf = DecisionTreeClassifier(criterion='entropy', max_depth=7, min_samples_leaf=

# Train Decision Tree Classifier
clf = clf.fit(X_train_pc,y_train)

#Predict the response for test dataset
y_pred = clf.predict(X_test_pc)

# Model Accuracy, how often is the classifier correct?
print("Accuracy:",metrics.accuracy_score(y_test, y_pred))

print(classification_report(y_test,y_pred, target_names=target_names))
```

Accuracy: 0.7495758398371225

	precision	recall	f1-score	support
Walking	0.98	0.96	0.97	537
Walking Upstairs	0.69	0.77	0.73	491
Walking Downstairs	0.79	0.71	0.75	532
Sitting	0.81	0.54	0.65	496
Standing	0.52	0.93	0.67	420
Laying	0.87	0.60	0.71	471
accuracy			0.75	2947
macro avg	0.78	0.75	0.75	2947
weighted avg	0.79	0.75	0.75	2947

Random Forest Classifier

In [114...

```
from sklearn.ensemble import RandomForestClassifier

# Create RF classifier object
clf = RandomForestClassifier(random_state=123)

# Train RF Classifier
clf = clf.fit(X_train_pc,y_train)

#Predict the response for test dataset
y_pred = clf.predict(X_test_pc)

# Model Accuracy, how often is the classifier correct?
print("Accuracy:",metrics.accuracy_score(y_test, y_pred))
```

```
print(classification_report(y_test,y_pred, target_names=target_names))
```

Accuracy: 0.827281981676281

	precision	recall	f1-score	support
Walking	0.99	0.99	0.99	537
Walking Upstairs	0.81	0.89	0.85	491
Walking Downstairs	0.90	0.81	0.85	532
Sitting	0.98	0.63	0.77	496
Standing	0.55	0.95	0.70	420
Laying	0.94	0.69	0.80	471
accuracy			0.83	2947
macro avg	0.86	0.83	0.83	2947
weighted avg	0.87	0.83	0.83	2947

Random Forest hyperparameter tuning: Grid Search

```
In [114...] params = {
    'n_estimators': [5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19],
    'max_depth': [2,3,4,5,6,7,8,9,10,11,12,13,14,15]
}

# Create a random forest classifier
rf = RandomForestClassifier()

# Instantiate the grid search model
grid_search = GridSearchCV(estimator=rf,
                           param_grid=params,
                           cv=4, n_jobs=-1, verbose=1, scoring = "accuracy")

# Fit the random search object to the data
grid_search.fit(X_train_pc, y_train)
```

Fitting 4 folds for each of 210 candidates, totalling 840 fits

```
Out[114...] GridSearchCV ⓘ ?
  estimator: RandomForestClassifier
    RandomForestClassifier ?
```

```
In [114...] grid_search.best_estimator_
```

```
Out[114...] RandomForestClassifier
RandomForestClassifier(max_depth=12, n_estimators=18)
```

```
In [114...] grid_search.best_score_
```

```
Out[114...] 0.8477965179542981
```

```
In [114... print(classification_report(y_test,y_pred, target_names=target_names))
```

	precision	recall	f1-score	support
Walking	0.99	0.99	0.99	537
Walking Upstairs	0.81	0.89	0.85	491
Walking Downstairs	0.90	0.81	0.85	532
Sitting	0.98	0.63	0.77	496
Standing	0.55	0.95	0.70	420
Laying	0.94	0.69	0.80	471
accuracy			0.83	2947
macro avg	0.86	0.83	0.83	2947
weighted avg	0.87	0.83	0.83	2947

Logistic Regression

```
In [115... # import the class
from sklearn.linear_model import LogisticRegression

# instantiate the model (using the default parameters)
logreg = LogisticRegression(random_state=16)

# fit the model with data
logreg.fit(X_train, y_train)

y_pred = logreg.predict(X_test)

print("Accuracy:", metrics.accuracy_score(y_test, y_pred))
```

Accuracy: 0.8476416694944011

```
In [115... print(classification_report(y_test,y_pred, target_names=target_names))
```

	precision	recall	f1-score	support
Walking	1.00	1.00	1.00	537
Walking Upstairs	0.63	0.98	0.77	491
Walking Downstairs	0.98	0.45	0.62	532
Sitting	0.99	0.88	0.93	496
Standing	0.74	1.00	0.85	420
Laying	0.99	0.81	0.89	471
accuracy			0.85	2947
macro avg	0.89	0.85	0.84	2947
weighted avg	0.89	0.85	0.84	2947

Logistic Regression hyperparameter tuning: Grid Search

```
In [115... from sklearn.model_selection import RepeatedStratifiedKFold

solvers = ['newton-cg', 'lbfgs', 'liblinear']
penalty = ['l2']
c_values = [100, 10, 1.0, 0.1, 0.01]
```

```

logreg = LogisticRegression(random_state=16)

# define grid search
grid = dict(solver=solvers,penalty=penalty,C=c_values)
cv = RepeatedStratifiedKfold(n_splits=10, n_repeats=3, random_state=1)

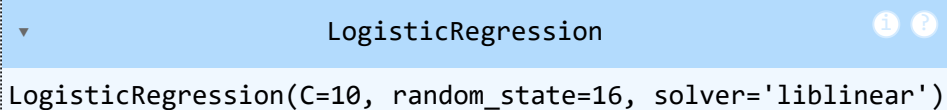
grid_search = GridSearchCV(estimator=logreg, param_grid=grid, n_jobs=-1, cv=cv,
grid_result = grid_search.fit(X_train, y_train)

# summarize results
print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))

```

Best: 0.986534 using {'C': 10, 'penalty': 'l2', 'solver': 'liblinear'}

In [117... grid_search.best_estimator_

Out[117... 

In [117... print(classification_report(y_test,y_pred, target_names=target_names))

	precision	recall	f1-score	support
Walking	0.99	1.00	1.00	537
Walking Upstairs	0.64	0.98	0.77	491
Walking Downstairs	0.98	0.49	0.65	532
Sitting	0.98	0.91	0.94	496
Standing	0.75	1.00	0.86	420
Laying	1.00	0.78	0.88	471
accuracy			0.85	2947
macro avg	0.89	0.86	0.85	2947
weighted avg	0.89	0.85	0.85	2947

SVM Classifier

In [119...

```

#Import svm model
from sklearn import svm

#Create a svm Classifier
#svmc = svm.SVC(kernel='linear', C=10) # Linear Kernel
svmc = svm.SVC(kernel='rbf')
#svmc = svm.SVC(kernel="poly", degree=3)

#Train the model using the training sets
svmc.fit(X_train, y_train)

#Predict the response for test dataset
y_pred = svmc.predict(X_test)

```

In [120...

```

# Model Accuracy: how often is the classifier correct?
print("Accuracy:",metrics.accuracy_score(y_test, y_pred))

```

```
print(classification_report(y_test,y_pred, target_names=target_names))
```

Accuracy: 0.8659653885307091

	precision	recall	f1-score	support
Walking	1.00	1.00	1.00	537
Walking Upstairs	0.69	0.98	0.81	491
Walking Downstairs	0.98	0.59	0.74	532
Sitting	0.99	0.85	0.92	496
Standing	0.72	1.00	0.84	420
Laying	0.99	0.80	0.88	471
accuracy			0.87	2947
macro avg	0.89	0.87	0.86	2947
weighted avg	0.90	0.87	0.86	2947

SVM hyperparameter tuning: Bayesian Optimisation

```
In [120... # https://sites.gatech.edu/omscs7641/2024/02/16/tutorial-on-hyperparameter-tuning/

from sklearn.model_selection import cross_val_score
from sklearn.metrics import classification_report
from GPyOpt.methods import BayesianOptimization
from skopt import BayesSearchCV, space, plots
from sklearn.svm import SVC

# Bayesian Search search space
search_spaces = {
    'C': (1e-2, 1e2, 'log-uniform'),
    'gamma': (1e-2, 1e2, 'log-uniform'),
    'kernel': ['rbf']
}

bayes_opt = BayesSearchCV(
    SVC(),
    search_spaces=search_spaces,
    n_iter=81,
    cv=5,
    n_jobs=4
)

bayes_opt.fit(X_train, y_train)
print(f'Best Params: {bayes_opt.best_params_}')

y_pred = bayes_opt.predict(X_test)
print(classification_report(y_test, y_pred))
```

Best Params: OrderedDict({'C': 10.250303216431895, 'gamma': 0.06465388719294979, 'kernel': 'rbf'})

	precision	recall	f1-score	support
LAYING	0.99	1.00	1.00	537
SITTING	0.62	0.98	0.76	491
STANDING	0.98	0.45	0.62	532
WALKING	1.00	0.82	0.90	496
WALKING_DOWNSTAIRS	0.68	1.00	0.81	420
WALKING_UPSTAIRS	1.00	0.77	0.87	471
accuracy			0.83	2947
macro avg	0.88	0.84	0.83	2947
weighted avg	0.89	0.83	0.83	2947

KNN

In [118...

```
from sklearn.neighbors import KNeighborsClassifier

# output classes: 'LAYING' 'SITTING' 'STANDING' 'WALKING' 'WALKING_DOWNSTAIR', t

knn = KNeighborsClassifier(n_neighbors=6)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)

print("Accuracy:", metrics.accuracy_score(y_test, y_pred))

print(classification_report(y_test,y_pred, target_names=target_names))
```

Accuracy: 0.8883610451306413

	precision	recall	f1-score	support
Walking	1.00	1.00	1.00	537
Walking Upstairs	0.80	0.94	0.87	491
Walking Downstairs	0.94	0.78	0.85	532
Sitting	0.90	0.84	0.87	496
Standing	0.75	0.93	0.83	420
Laying	0.97	0.84	0.90	471
accuracy			0.89	2947
macro avg	0.89	0.89	0.89	2947
weighted avg	0.90	0.89	0.89	2947

SUMMARY

Best model accuracies are summarised below.

Decision Tree (default hyperparameters): accuracy = 0.73

Decision Tree (criterion='entropy', max_depth=7, min_samples_leaf=20,): accuracy = 0.75

Random Forest (default hyperparameters): accuracy = 0.83

Logistic Regression (default hyperparameters): accuracy = 0.85

Support Vector Machine (default hyperparameters): accuracy = 0.85

KNN (k=6): accuracy = 0.89

DISCUSSION

I have trained several models: DT, RF, LogReg, SVM, and KNN to classify human activities from the human-activity-recognition-with-smarphone dataset. The train dataset consists of 562 features and 7352 samples, while the test dataset is comprised of 2947 samples.

With such a large number of input features, I thought that dimensionality reduction might be beneficial.

I found that decision trees and random forests gave the best accuracies with models trained using the principal component datasets, while the remaining models (LogReg, SVM and KNN) gave better accuracies without dimensionality reduction.

Although my accuracy scores aren't competition winning, they are fairly good. Given more time, I would have liked to try neural network models and spent a bit more time fine tuning by best performing models.⁶

In []: